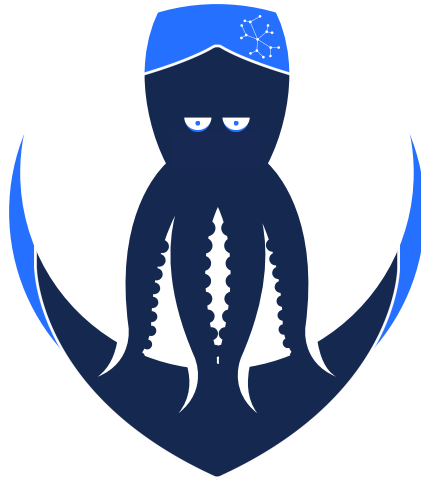




Boss Bridge Initial Audit Report

Version 0.1

@aimanubayd

September 18, 2026

Boss Bridge Audit Report

Aiman Ubayd

Sep 7, 2026

Boss Bridge Audit Report

Prepared by: Aiman_Ubayd Lead Auditor: Aiman Ubayd

Assisting Auditors:

- None

Table of contents

See table

- Boss Bridge Audit Report
- Table of contents
- About Aiman Ubayd
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
- Protocol Summary
 - Roles
- Executive Summary

- Issues found
- Findings
 - High
 - * [H-1] Users who give tokens approvals to `L1BossBridge` may have those asset stolen (`from` address in `depositTokensToL2` is caller-controlled -> attacker moves any approved user's tokens into the vault and claims them on L2)
 - * [H-2] Calling `depositTokensToL2` from the Vault contract to the Vault contract allows infinite minting of unbacked tokens (Vault's infinite approval to the bridge combined with a caller-controlled `from` -> attacker self-transfers vault funds to trigger unlimited unbacked L2 mints)
 - * [H-3] Lack of replay protection in `withdrawTokensToL1` allows withdrawals by signature to be replayed (Operator withdrawal signatures contain no nonce or single-use marker -> a valid signature can be resubmitted repeatedly to drain the vault)
 - * [H-4] `L1BossBridge::sendToL1` allowing arbitrary calls enables users to call `L1Vault::approveTo` and give themselves infinite allowance of vault funds (Operator-signed message can target arbitrary contracts/calldata -> attacker crafts a signed call to `L1Vault::approveTo` and drains the vault)
 - * [H-5] `CREATE` opcode does not work on zksync era (Token deployment relies on the `CREATE` opcode via `new` -> `TokenFactory::deployToken` fails or behaves unexpectedly if the bridge is ever deployed to zkSync Era)
 - * [H-6] `L1BossBridge::depositTokensToL2`'s `DEPOSIT_LIMIT` check allows contract to be DoS'd (Deposit cap is enforced against the vault's raw token balance, which anyone can inflate directly -> attacker permanently blocks all future legitimate deposits)
 - * [H-7] The `L1BossBridge::withdrawTokensToL1` function has no validation on the withdrawal amount being the same as the deposited amount in `L1BossBridge::depositTokensToL2`, allowing attacker to withdraw more funds than deposited (No on-chain link between a user's deposited amount and their withdrawal amount -> withdrawal size is enforced only by off-chain operator judgement, not by contract logic)
 - * [H-8] `TokenFactory::deployToken` locks tokens forever (Newly deployed token's initial supply is minted with no mechanism to move it out of `TokenFactory` -> minted tokens become permanently inaccessible)
 - Medium
 - * [M-1] Withdrawals are prone to unbounded gas consumption due to return bombs (Low-level call forwards all available gas and copies all returndata -> a malicious withdrawal target can force excessive gas expenditure on the caller)

- Low
 - * [L-1] Lack of event emission during withdrawals and sending tokens to L1 (No `Withdrawal`-style event emitted on successful `sendToL1/withdrawTokensToL1` calls -> off-chain monitoring cannot detect or alert on withdrawal activity)
 - * [L-2] `TokenFactory::deployToken` can create multiple token with same `symbol` (No uniqueness check on token `symbol` -> duplicate-symbol tokens can be deployed, creating ambiguity for integrators and users)
 - * [L-3] Unsupported opcode `PUSH0` (Compiler may target Shanghai's `PUSH0` opcode by default -> deployment fails on chains/EVM versions that do not yet support it)
- Informational
 - * [I-1] Insufficient test coverage (Coverage gaps in `L1Vault.sol` and partial branch coverage in `L1BossBridge.sol` -> untested code paths increase the risk that further bugs go undetected)

About Aiman Ubayd

Aiman Ubayd is a smart contract auditor and security researcher. He researches how EVM & DeFi protocols fail, tracing exploit paths in protocol logic, economics, and trust boundaries. He combines manual security research with fuzzing, invariant testing, and formal verification, backed by AI-powered tooling and AI agents for web3 security analysis.

- **X (Twitter):** [<https://x.com/aimanubayd>]
- **GitHub:** [<https://github.com/aimanubayd>]
- **LinkedIn:** [<https://linkedin.com/in/aimanubayd>]
- **Contact:** [aimanubayd@gmail.com]

Disclaimer

Aiman Ubayd makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

Audit Details

The findings described in this document correspond the following commit hash:

```
1 07af21653ab3e8a8362bf5f63eb058047f562375
```

Scope

```
1 #-- src
2 |    #-- L1BossBridge.sol
3 |    #-- L1Token.sol
4 |    #-- L1Vault.sol
5 |    #-- TokenFactory.sol
```

Protocol Summary

The Boss Bridge is a bridging mechanism to move an ERC20 token (the “Boss Bridge Token” or “BBT”) from L1 to an L2 the development team claims to be building. Because the L2 part of the bridge is under construction, it was not included in the reviewed codebase.

The bridge is intended to allow users to deposit tokens, which are to be held in a vault contract on L1. Successful deposits should trigger an event that an off-chain mechanism is in charge of detecting to mint the corresponding tokens on the L2 side of the bridge.

Withdrawals must be approved operators (or “signers”). Essentially they are expected to be one or more off-chain services where users request withdrawals, and that should verify requests before signing the

data users must use to withdraw their tokens. It's worth highlighting that there's little-to-no on-chain mechanism to verify withdrawals, other than the operator's signature. So the Boss Bridge heavily relies on having robust, reliable and always available operators to approve withdrawals. Any rogue operator or compromised signing key may put at risk the entire protocol.

Roles

- Bridge owner: can pause and unpause withdrawals in the [L1BossBridge](#) contract. Also, can add and remove operators. Rogue owners or compromised keys may put at risk all bridge funds.
- User: Accounts that hold BBT tokens and use the [L1BossBridge](#) contract to deposit and withdraw them.
- Operator: Accounts approved by the bridge owner that can sign withdrawal operations. Rogue operators or compromised keys may put at risk all bridge funds.

Executive Summary

Issues found

Severity	Number of issues found
High	8
Medium	1
Low	3
Info	1
Gas	0
Total	13

Findings

High

[H-1] Users who give tokens approvals to L1BossBridge may have those assest stolen (from address in depositTokensToL2 is caller-controlled -> attacker moves any approved user's tokens into the vault and claims them on L2)

Description: The `depositTokensToL2` function allows anyone to call it with a `from` address of any account that has approved tokens to the bridge. Nothing in the function restricts `from` to being `msg.sender`.

```
1 function depositTokensToL2(address from, address l2Recipient, uint256
  amount) external whenNotPaused {
2   if (token.balanceOf(address(vault)) + amount > DEPOSIT_LIMIT) {
3     revert L1BossBridge__DepositLimitReached();
4   }
5   token.transferFrom(from, address(vault), amount);
6   emit Deposit(from, l2Recipient, amount);
7 }
```

Impact: An attacker can move tokens out of any victim account whose token allowance to the bridge is greater than zero. This moves the tokens into the bridge vault and assigns them to the attacker's own address in L2 (by setting an attacker-controlled `l2Recipient`), resulting in the victim losing their tokens on L1 while the attacker receives the corresponding minted tokens on L2.

Proof of Concept:

```
1 function testCanMoveApprovedTokensOfOtherUsers() public {
2   vm.prank(user);
3   token.approve(address(tokenBridge), type(uint256).max);
4
5   uint256 depositAmount = token.balanceOf(user);
6   vm.startPrank(attacker);
7   vm.expectEmit(address(tokenBridge));
8   emit Deposit(user, attackerInL2, depositAmount);
9   tokenBridge.depositTokensToL2(user, attackerInL2, depositAmount);
10
11   assertEq(token.balanceOf(user), 0);
12   assertEq(token.balanceOf(address(vault)), depositAmount);
13   vm.stopPrank();
14 }
```

Recommended Mitigation: Modify the `depositTokensToL2` function so that the caller cannot specify a `from` address.

```
1 - function depositTokensToL2(address from, address l2Recipient, uint256
    amount) external whenNotPaused {
2 + function depositTokensToL2(address l2Recipient, uint256 amount)
    external whenNotPaused {
3     if (token.balanceOf(address(vault)) + amount > DEPOSIT_LIMIT) {
4         revert L1BossBridge__DepositLimitReached();
5     }
6 - token.transferFrom(from, address(vault), amount);
7 + token.transferFrom(msg.sender, address(vault), amount);
8
9     // Our off-chain service picks up this event and mints the
        corresponding tokens on L2
10 - emit Deposit(from, l2Recipient, amount);
11 + emit Deposit(msg.sender, l2Recipient, amount);
12 }
```

[H-2] Calling `depositTokensToL2` from the Vault contract to the Vault contract allows infinite minting of unbacked tokens (Vault's infinite approval to the bridge combined with a caller-controlled `from` -> attacker self-transfers vault funds to trigger unlimited unbacked L2 mints)

Description: The `depositTokensToL2` function allows the caller to specify the `from` address, from which tokens are taken. Because the vault grants infinite approval to the bridge already (as can be seen in the contract's constructor), it's possible for an attacker to call the `depositTokensToL2` function and transfer tokens from the vault to the vault itself.

Impact: This would allow the attacker to trigger the `Deposit` event any number of times, presumably causing the minting of unbacked tokens in L2, without the vault's real token balance ever decreasing. Additionally, the attacker could set themselves as the `l2Recipient`, minting all of the resulting L2 tokens to themselves.

Proof of Concept:

```
1 function testCanTransferFromVaultToVault() public {
2     vm.startPrank(attacker);
3
4     // assume the vault already holds some tokens
5     uint256 vaultBalance = 500 ether;
6     deal(address(token), address(vault), vaultBalance);
7
8     // Can trigger the `Deposit` event self-transferring tokens in the
        vault
9     vm.expectEmit(address(tokenBridge));
10    emit Deposit(address(vault), address(vault), vaultBalance);
```

```
11     tokenBridge.depositTokensToL2(address(vault), address(vault),  
12         vaultBalance);  
13     // Any number of times  
14     vm.expectEmit(address(tokenBridge));  
15     emit Deposit(address(vault), address(vault), vaultBalance);  
16     tokenBridge.depositTokensToL2(address(vault), address(vault),  
17         vaultBalance);  
18     vm.stopPrank();  
19 }
```

Recommended Mitigation: As suggested in H-1, consider modifying the `depositTokensToL2` function so that the caller cannot specify a `from` address.

[H-3] Lack of replay protection in `withdrawTokensToL1` allows withdrawals by signature to be replayed (Operator withdrawal signatures contain no nonce or single-use marker -> a valid signature can be resubmitted repeatedly to drain the vault)

Description: Users who want to withdraw tokens from the bridge can call the `sendToL1` function, or the wrapper `withdrawTokensToL1` function. These functions require the caller to send along some withdrawal data signed by one of the approved bridge operators. However, the signatures do not include any kind of replay-protection mechanism (e.g., nonces).

Impact: A valid signature from any bridge operator can be reused by any attacker to continue executing withdrawals until the vault is completely drained, since nothing on-chain marks a signature as “already used.”

Proof of Concept:

```
1  function testCanReplayWithdrawals() public {  
2      // Assume the vault already holds some tokens  
3      uint256 vaultInitialBalance = 1000e18;  
4      uint256 attackerInitialBalance = 100e18;  
5      deal(address(token), address(vault), vaultInitialBalance);  
6      deal(address(token), address(attacker), attackerInitialBalance);  
7  
8      // An attacker deposits tokens to L2  
9      vm.startPrank(attacker);  
10     token.approve(address(tokenBridge), type(uint256).max);  
11     tokenBridge.depositTokensToL2(attacker, attackerInL2,  
12         attackerInitialBalance);  
13  
14     // Operator signs withdrawal.  
15     (uint8 v, bytes32 r, bytes32 s) =
```

```
15     _signMessage(_getTokenWithdrawalMessage(attacker,  
16               attackerInitialBalance), operator.key);  
17     // The attacker can reuse the signature and drain the vault.  
18     while (token.balanceOf(address(vault)) > 0) {  
19         tokenBridge.withdrawTokensToL1(attacker, attackerInitialBalance  
20             , v, r, s);  
21     }  
22     assertEq(token.balanceOf(address(attacker)), attackerInitialBalance  
23         + vaultInitialBalance);  
24     assertEq(token.balanceOf(address(vault)), 0);  
25 }
```

Recommended Mitigation: Consider redesigning the withdrawal mechanism so that it includes replay protection, such as a per-withdrawal nonce that is checked and invalidated on-chain (or by binding each signature to a unique withdrawal identifier that can only be executed once).

[H-4] L1BossBridge::sendToL1 allowing arbitrary calls enables users to call L1Vault::approveTo and give themselves infinite allowance of vault funds (Operator-signed message can target arbitrary contracts/calldata -> attacker crafts a signed call to L1Vault::approveTo and drains the vault)

Description: The `L1BossBridge` contract includes the `sendToL1` function that, if called with a valid signature by an operator, can execute arbitrary low-level calls to any given target. Because there's no restriction on either the target or the calldata, this call could be used by an attacker to execute sensitive functions of contracts the bridge owns — for example, the `L1Vault` contract, which is owned by `L1BossBridge` and exposes an `approveTo` function.

Impact: An attacker could submit a call that targets the vault and executes its `approveTo` function, passing an attacker-controlled address to increase its allowance to the maximum value. This would allow the attacker to completely drain the vault. This attack's likelihood depends on the sophistication of the off-chain validation performed by operators before signing, but per the available documentation, the only validation performed is that “the account submitting the withdrawal has first originated a successful deposit in the L1 part of the bridge” — as the PoC below shows, that check alone does not prevent the attack, since it can be satisfied trivially with a zero-value deposit.

Proof of Concept:

```
1 function testCanCallVaultApproveFromBridgeAndDrainVault() public {  
2     uint256 vaultInitialBalance = 1000e18;  
3     deal(address(token), address(vault), vaultInitialBalance);  
4 }
```

```
5      // An attacker deposits tokens to L2. We do this under the
6      // assumption that the
7      // bridge operator needs to see a valid deposit tx to then allow us
8      // to request a withdrawal.
9      vm.startPrank(attacker);
10     vm.expectEmit(address(tokenBridge));
11     emit Deposit(address(attacker), address(0), 0);
12     tokenBridge.depositTokensToL2(attacker, address(0), 0);
13
14     // Under the assumption that the bridge operator doesn't validate
15     // bytes being signed
16     bytes memory message = abi.encode(
17         address(vault), // target
18         0, // value
19         abi.encodeCall(L1Vault.approveTo, (address(attacker), type(
20             uint256).max)) // data
21     );
22     (uint8 v, bytes32 r, bytes32 s) = _signMessage(message, operator.
23         key);
24
25     tokenBridge.sendToL1(v, r, s, message);
26     assertEq(token.allowance(address(vault), attacker), type(uint256).
27         max);
28     token.transferFrom(address(vault), attacker, token.balanceOf(
29         address(vault)));
30 }
```

Recommended Mitigation: Consider disallowing attacker-controlled external calls to sensitive components of the bridge, such as the `L1Vault` contract — for example, by restricting `sendToL1`'s allowed target addresses to an explicit allow-list that excludes the vault and any other privileged bridge contracts.

[H-5] CREATE opcode does not work on zksync era (Token deployment relies on the CREATE opcode via `new` -> `TokenFactory::deployToken` fails or behaves unexpectedly if the bridge is ever deployed to zkSync Era)

Description: `TokenFactory::deployToken` deploys new token contracts using Solidity's `new` keyword, which compiles down to the `CREATE` opcode. zkSync Era's EVM-compatible environment handles contract deployment differently from standard EVM chains — deployed bytecode must be registered as a known “factory dependency” ahead of time, and `CREATE`/`CREATE2` do not behave identically to their Ethereum L1 counterparts. Given that the protocol's stated goal is to bridge tokens toward an L2 (and zkSync Era is a common target for such bridges), this is a realistic deployment target worth accounting for.

Impact: If `TokenFactory` (or any other contract relying on `new` for deployment) is deployed to zkSync Era without adjustment, token deployment via `deployToken` may revert entirely or fail to produce a contract at the address the calling code expects, breaking the token-creation flow that the rest of the bridge depends on.

Proof of Concept: N/A — this is a chain-compatibility observation based on documented differences between zkSync Era's compiler/VM and standard EVM `CREATE` semantics, rather than a bug reproducible on the currently targeted network.

Recommended Mitigation: If zkSync Era (or another zk-EVM with non-standard `CREATE` handling) is a genuine deployment target, use zkSync's dedicated compiler (`zksolc`) and deployment tooling, which correctly registers factory dependencies, or use `CREATE2` with explicitly precomputed bytecode hashes as required by that environment. If zkSync Era is not an intended target, state this explicitly in the protocol's documentation to avoid an unsafe deployment.

[H-6] L1BossBridge::depositTokensToL2's DEPOSIT_LIMIT check allows contract to be DoS'd (Deposit cap is enforced against the vault's raw token balance, which anyone can inflate directly -> attacker permanently blocks all future legitimate deposits)

Description: `depositTokensToL2` enforces its deposit cap by checking the vault's current raw token balance:

```
1 if (token.balanceOf(address(vault)) + amount > DEPOSIT_LIMIT) {  
2     revert L1BossBridge__DepositLimitReached();  
3 }
```

Because this check reads `token.balanceOf(address(vault))` directly rather than tracking deposits through an internal accounting variable, anyone can increase the vault's token balance without ever calling `depositTokensToL2` — simply by calling `token.transfer(address(vault), amount)` directly on the token contract.

Impact: An attacker can transfer just enough tokens directly to the vault to push its balance up to (or past) `DEPOSIT_LIMIT`, without needing to go through the bridge's deposit flow at all. Once the vault's balance is at or above the limit, every legitimate call to `depositTokensToL2` will revert, permanently denying service to all users wishing to deposit — a low-cost, difficult-to-reverse denial-of-service.

Proof of Concept: 1. Attacker calls `token.transfer(address(vault), DEPOSIT_LIMIT)` directly (no interaction with `L1BossBridge` required). 2. The vault's `token.balanceOf(address(vault))` is now at `DEPOSIT_LIMIT`. 3. Any subsequent call to `depositTokensToL2`

, even for a legitimate user depositing a small amount, causes `token.balanceOf(address(vault)) + amount > DEPOSIT_LIMIT` to evaluate true, reverting the deposit.

Recommended Mitigation: Track total deposits using an internal accounting variable updated only within `depositTokensToL2`, rather than reading the vault's raw token balance, so that direct token transfers to the vault cannot influence the deposit-limit check.

[H-7] The `L1BossBridge::withdrawTokensToL1` function has no validation on the withdrawal amount being the same as the deposited amount in `L1BossBridge::depositTokensToL2`, allowing attacker to withdraw more funds than deposited (No on-chain link between a user's deposited amount and their withdrawal amount -> withdrawal size is enforced only by off-chain operator judgement, not by contract logic)

Description: `withdrawTokensToL1` (and the underlying `sendToL1`) validate only that the withdrawal message was signed by an approved operator — there is no on-chain check comparing the requested withdrawal amount against how much that account actually deposited via `depositTokensToL2`. All enforcement of “you may only withdraw what you deposited” happens entirely off-chain, in whatever validation logic the operator chooses to apply before signing.

Impact: If an operator signs a withdrawal message for an amount exceeding what the requesting account ever deposited — whether due to a bug, a compromised operator key, or (as shown in [H-4]) insufficiently strict off-chain validation logic — the contract itself has no way to detect or reject this, and will process the withdrawal for the full signed amount, up to the entire balance of the vault.

Proof of Concept: As demonstrated in [H-4]'s proof of concept, a user can trigger a valid `Deposit` event with an amount of 0, which is enough to satisfy an operator's stated policy of “the account submitting the withdrawal has first originated a successful deposit.” Nothing in `withdrawTokensToL1` itself then constrains the withdrawal amount to match any specific prior deposit, meaning the contract's own logic places no upper bound tied to actual deposited value.

Recommended Mitigation: Track each account's net deposited (and not-yet-withdrawn) balance on-chain, and have `withdrawTokensToL1` validate that the requested withdrawal amount does not exceed that account's recorded balance, rather than relying solely on the operator's off-chain signature as the only check.

[H-8] TokenFactory::deployToken locks tokens forever (Newly deployed token's initial supply is minted with no mechanism to move it out of TokenFactory -> minted tokens become permanently inaccessible)

Description: `TokenFactory::deployToken` deploys a new `L1Token`-style contract and, as part of construction, mints its initial supply. However, that minted supply is assigned to the `TokenFactory` contract itself (or otherwise ends up controlled by it), and `TokenFactory` exposes no function to transfer, withdraw, or otherwise move that minted balance to any other address.

Impact: Every token deployed through `deployToken` has its entire initial supply permanently and irrecoverably locked inside the `TokenFactory` contract, since there is no code path by which that balance can ever be transferred out. This renders the newly deployed token practically useless unless additional supply can be minted elsewhere.

Proof of Concept: Call `TokenFactory::deployToken` for a new token, then inspect the resulting token's balance for `address(tokenFactory)` — it will hold the full initial minted supply, and no function on `TokenFactory` exists to transfer that balance to any other account.

Recommended Mitigation: Modify `deployToken` to mint the initial supply directly to an intended recipient (e.g., the caller, or an address passed as a parameter), or add a dedicated function allowing the factory owner to withdraw/distribute tokens held by the factory after deployment.

Medium

[M-1] Withdrawals are prone to unbounded gas consumption due to return bombs (Low-level call forwards all available gas and copies all returndata -> a malicious withdrawal target can force excessive gas expenditure on the caller)

Description: During withdrawals, the L1 part of the bridge executes a low-level call to an arbitrary target, passing all available gas. While this works fine for regular targets, it may not for adversarial ones. In particular, a malicious target may drop a return bomb — returning a large amount of returndata from the call, which Solidity automatically copies into memory, incurring expensive memory-expansion gas costs.

Impact: Callers unaware of this risk may not set the transaction's gas limit sensibly, and could be tricked into spending significantly more ETH than necessary to execute the call, or have their transaction run out of gas and revert entirely despite the underlying operation otherwise being valid.

Proof of Concept: N/A — this is a gas-griefing pattern arising from any low-level call that forwards all available gas to an untrusted, attacker-controlled target and copies its full returndata, rather than a specific reproducible exploit against a fixed balance.

Recommended Mitigation: If the external call's returndata is not needed, modify the call to avoid copying any of it. This can be done via a custom low-level call implementation that caps the copied returndata size, or by reusing an existing library such as `ExcessivelySafeCall`.

Low

[L-1] Lack of event emission during withdrawals and sending tokens to L1 (No `Withdrawal`-style event emitted on successful `sendToL1`/`withdrawTokensToL1` calls -> off-chain monitoring cannot detect or alert on withdrawal activity)

Description: Neither the `sendToL1` function nor the `withdrawTokensToL1` function emit an event when a withdrawal operation is successfully executed.

Impact: This prevents off-chain monitoring mechanisms from tracking withdrawals and raising alerts on suspicious scenarios (for example, an unusually large withdrawal, or a burst of withdrawals shortly after deployment), reducing the bridge operators' and users' ability to detect an ongoing attack such as those described in [H-3] and [H-4] while it is still in progress.

Proof of Concept: Call `withdrawTokensToL1` (or `sendToL1`) successfully and observe that no event is emitted in the transaction logs corresponding to the completed withdrawal.

Recommended Mitigation: Modify the `sendToL1` function to include a new event that is always emitted upon completing withdrawals, capturing at minimum the recipient, amount, and target/calldata involved.

[L-2] `TokenFactory::deployToken` can create multiple token with same symbol (No uniqueness check on token symbol -> duplicate-symbol tokens can be deployed, creating ambiguity for integrators and users)

Description: `TokenFactory::deployToken` accepts a `symbol` parameter used to configure the newly deployed token, but does not check whether a token with that same symbol has already been deployed through the factory.

Impact: Multiple, entirely distinct token contracts can be deployed with identical symbols. This can confuse users, wallets, and third-party integrators that identify tokens by symbol rather than address,

potentially leading someone to interact with the wrong token contract under the mistaken belief it is the original.

Proof of Concept: Call `TokenFactory::deployToken` twice with the same `symbol` argument (but different underlying parameters/addresses) and observe that both calls succeed, producing two separate token contracts sharing the same symbol.

Recommended Mitigation: Track previously used symbols in a mapping and revert `deployToken` if the requested symbol has already been deployed, or otherwise document clearly that symbol uniqueness is not enforced and must be verified off-chain by integrators.

[L-3] Unsupported opcode PUSH0 (Compiler may target Shanghai's PUSH0 opcode by default -> deployment fails on chains/EVM versions that do not yet support it)

Description: Depending on the Solidity compiler version and configured EVM target used to build this project, contracts may be compiled to include the `PUSH0` opcode, introduced in the Shanghai upgrade. Not all chains the bridge might realistically be deployed to (including some L2s and zk-EVMs, relevant given [H-5]'s zkSync Era consideration) support this opcode.

Impact: If contracts are compiled targeting an EVM version that emits `PUSH0` and then deployed to a chain that does not yet support that opcode, deployment will fail outright, or the deployed bytecode may behave unpredictably depending on how the target chain handles the unrecognized opcode.

Proof of Concept: N/A — this is a build/deployment-configuration observation rather than a runtime exploit; verifying it requires checking the project's `foundry.toml`/compiler settings for the configured EVM version and confirming it against the actual capabilities of every intended deployment target.

Recommended Mitigation: Explicitly pin the compiler's target EVM version (e.g., `paris` instead of `shanghai` or later) in the project's build configuration to avoid emitting `PUSH0`, unless every intended deployment chain is confirmed to support it.

Informational

[I-1] Insufficient test coverage (Coverage gaps in `L1Vault.sol` and partial branch coverage in `L1BossBridge.sol` -> untested code paths increase the risk that further bugs go undetected)

Description: Running the project's test suite with coverage reporting shows incomplete coverage across the in-scope contracts, most notably `L1Vault.sol` at 0% coverage across all metrics.

```

1 Running tests...
2 | File | % Lines | % Statements | % Branches
3 | ----- | ----- | ----- |
4 | src/L1BossBridge.sol | 86.67% (13/15) | 90.00% (18/20) | 83.33% (5/6)
5 | src/L1Vault.sol | 0.00% (0/1) | 0.00% (0/1) | 100.00%
6 | src/TokenFactory.sol | 100.00% (4/4) | 100.00% (4/4) | 100.00%
7 | Total | 85.00% (17/20) | 88.00% (22/25) | 83.33% (5/6)

```

Impact: Untested code paths, particularly the completely uncovered `L1Vault.sol`, increase the likelihood that behavioral bugs or regressions in that contract go undetected by the project's own test suite, both now and in future changes.

Proof of Concept: N/A — see the coverage table above, generated via the project's coverage tooling.

Recommended Mitigation: Aim to get test coverage up to over 90% for all files, with particular priority given to adding tests for `L1Vault.sol`, which currently has none.